

《Attention Is All You Need》深度解析报告

执行摘要

《Attention Is All You Need》提出了 **Transformer**：一种**完全基于注意力机制**的序列到序列（seq2seq）架构，**不再依赖RNN的递归计算与CNN的卷积堆叠**，从而在机器翻译等序列建模任务上获得了更强的并行化能力，并在当时的WMT14英德/英法翻译上取得了领先结果。¹

原论文由Ashish Vaswani²、Noam Shazeer³、Niki Parmar⁴、Jakob Uszkoreit⁵、Llion Jones⁶、Aidan N. Gomez⁷、Łukasz Kaiser⁸、Illia Polosukhin⁹ 等作者完成，作者当时主要来自Google¹⁰ 旗下研究团队Google Brain¹¹ 与Google Research¹²，并包含University of Toronto¹³ 研究者。¹⁴

论文的核心贡献可以概括为三点：（1）用**多头自注意力**（Multi-Head Self-Attention）作为主要信息交互机制，在每层实现全局依赖建模；（2）用**缩放点积注意力**（Scaled Dot-Product Attention）稳定训练，并通过**掩码机制**支持自回归解码；（3）提出**正弦/余弦位置编码**解决无递归/无卷积时的序列位置信息注入，同时结合残差连接与层归一化使深层堆叠可训练。¹⁵

Transformer随后成为现代NLP与大模型的基础积木：例如BERT以“编码器堆叠”为主进行双向预训练，GPT系列以“解码器堆叠”为主进行自回归预训练；而针对长序列的Transformer-XL、Longformer以及更高效的稀疏/线性注意力方法，也多围绕“注意力的计算与记忆瓶颈”展开改进。¹⁶

背景与动机

在Transformer之前，主流seq2seq模型多采用“编码器—解码器”框架，其中编码器/解码器通常由**RNN（含LSTM/GRU）或CNN**构成，并辅以注意力机制连接编码器与解码器。¹⁷

但这类架构面临三个结构性问题（论文在“Why Self-Attention”部分以三个desiderata总结）：

第一，**并行化困难**：RNN需要沿时间步串行计算，长序列时训练效率受限；第二，**长程依赖路径长**：依赖跨越距离越远，信息与梯度传播路径越长，学习难度越大；第三，**计算复杂度与硬件效率的权衡**：卷积需要堆叠层数才能扩大感受野，RNN虽每步计算较集中但难并行，整体吞吐受限。¹⁸

Transformer的关键动机在于：如果我们把“序列位置之间的交互”从“递归状态传递”或“局部卷积聚合”改为**一次性（每层）全局可学习的加权聚合（attention）**，则可以：

- 让一个样本内的所有token表示在同一层中并行更新（矩阵乘法友好）；
- 把任意两位置之间的信息路径缩短到常数级（理想情况下为1）；
- 通过多头机制在不同子空间捕捉多种关系模式。¹⁹

需要强调：Transformer不是“抛弃注意力”，而是把注意力从“RNN/CNN的辅助模块”提升为“核心计算单元”。论文摘要与引言明确提出该架构“dispensing with recurrence and convolutions entirely”。²⁰

模型总体架构

Transformer整体是标准的**编码器—解码器**结构：编码器将输入序列映射为一组上下文表征（memory）；解码器在自回归生成时，既对已生成的前缀做masked self-attention，又通过encoder-decoder attention读取编码器memory，最终输出下一个token分布。编码器与解码器各由 **N=6层**相同结构堆叠（base与big均如此），这是论文给出的默认配置。²¹

```

flowchart LR
    A[输入tokens] --> B[Embedding × sqrt(d_model)]
    B --> C[+ Positional Encoding]
    C --> E[Encoder Layer × N]
    E --> M[Encoder Memory]

    D[目标tokens(右移)] --> F[Embedding × sqrt(d_model)]
    F --> G[+ Positional Encoding]
    G --> H[Decoder Layer × N]
    H --> I[Linear]
    I --> J[Softmax 输出分布]

    M --> H

```

图1: Transformer总体数据流（编码器产生memory，解码器在masked self-attn与cross-attn中使用）。 22

编码器层 (Encoder Layer) 包含两个子层：

- 子层1: Multi-Head **Self-Attention** (Q=K=V均来自上一层输出) ；
- 子层2: Position-wise **Feed-Forward Network (FFN)** 。 22

解码器层 (Decoder Layer) 包含三个子层（比编码器多一个cross-attn子层）：

- 子层1: Multi-Head **Masked Self-Attention** (防止看未来) ；
- 子层2: Multi-Head **Encoder-Decoder Attention** (Q来自解码器, K/V来自编码器memory) ；
- 子层3: Position-wise **FFN**。 23

残差连接与层归一化 (Post-LN)：每个子层外包一条残差，并在相加后做LayerNorm：

$$\text{LayerNorm}(x + \text{Sublayer}(x)).$$

论文明确说明 “residual connection around each sub-layer, followed by layer normalization” 。 24

嵌入、Softmax与权重共享：输入/输出token先映射到维度为 d_{model} 的向量；解码器输出再经线性层+softmax得到词表概率。论文共享两处embedding权重（源/目标embedding与pre-softmax线性变换权重），并在embedding处乘 $\sqrt{d_{\text{model}}}$ 以匹配尺度。 25

位置编码 (Positional Encoding)：因为没有递归/卷积，必须显式注入位置信息；位置编码与embedding同维度相加，放在编码器/解码器堆叠底部。论文采用固定的正弦/余弦位置编码（也实验了可学习位置embedding，效果接近）。 26

补充：中文高质量教材《动手学深度学习》对编码器/解码器层结构、残差连接与LayerNorm的维度约束、以及多头注意力—FFN的堆叠方式给出了较系统的解释，可作为对照阅读。 27

关键数学推导与机制

本节把Transformer最关键的数学组件拆开，给出符号表、逐步推导与“工程可落地”的计算流程。

符号与形状说明表

符号	含义	典型形状（单样本，不含batch维）
n	序列长度	标量
d_{model}	模型隐藏维度	标量（base=512）
h	注意力头数	标量（base=8）
d_k, d_v	每个头的key/query与value维度	标量（base中 $d_k = d_v = d_{\text{model}}/h = 64$ ）
X	输入序列表示（某层输入）	$\mathbb{R}^{n \times d_{\text{model}}}$
W^Q, W^K, W^V	线性投影矩阵（单头写法）	$\mathbb{R}^{d_{\text{model}} \times d_k}, \mathbb{R}^{d_{\text{model}} \times d_k}, \mathbb{R}^{d_{\text{model}} \times d_v}$
Q, K, V	Query/Key/Value矩阵	$\mathbb{R}^{n \times d_k}, \mathbb{R}^{n \times d_k}, \mathbb{R}^{n \times d_v}$
A	注意力权重矩阵（softmax后）	$\mathbb{R}^{n \times n}$
M	掩码矩阵（允许=1/禁止=0）	$\mathbb{R}^{n \times n}$ 或广播形状

表中base配置来自论文对Multi-Head Attention的设定（ $h = 8, d_k = d_v = d_{\text{model}}/h$ ）。 28

自注意力：从“加权求和”到矩阵形式

注意力的核心是：对每个位置 i ，输出是对所有value向量的加权和

$$\text{out}_i = \sum_{j=1}^n \alpha_{ij} v_j, \quad \sum_j \alpha_{ij} = 1.$$

权重 α_{ij} 由query与key的相似度决定，并经过softmax归一化。论文采用点积相似度（dot-product / multiplicative），并给出矩阵打包后的计算式。 29

把所有位置的query、key、value堆成矩阵：

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

其中 Q, K, V 分别为 $\mathbb{R}^{n \times d_k}$ 、 $\mathbb{R}^{n \times d_k}$ 、 $\mathbb{R}^{n \times d_v}$ 。 29

相似度（未softmax）矩阵是

$$S = QK^\top \in \mathbb{R}^{n \times n},$$

其中 $S_{ij} = q_i^\top k_j$ 。 29

缩放点积注意力：为什么要除以 $\sqrt{d_k}$

论文最终使用的注意力计算为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

这是公式(1)。 30

缩放因子 $\frac{1}{\sqrt{d_k}}$ 的动机：论文给出一个直观统计解释：若 q 与 k 各维独立、均值0方差1，则点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的方差随 d_k 线性增长 ($\text{Var}(q \cdot k) = d_k$)，导致 $QK^\top$ 的数值幅度随维度变大而变大，从而把softmax推向梯度很小的饱和区；除以 $\sqrt{d_k}$ 可把尺度拉回“更稳定”的范围。 29

用“方差归一”的角度把这个推导写得更明确：

1. 假设 $q_i, k_i \sim (0, 1)$ 独立，则 $\mathbb{E}[q_i k_i] = 0$, $\text{Var}(q_i k_i) = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1$ 。 29
2. 点积是 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ ，独立求和方差相加： $\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$ 。 29
3. 因此 $(q \cdot k) / \sqrt{d_k}$ 的方差约为1，更利于softmax处于“有梯度”的区域。 29

这也是为什么很多“Transformer实现/讲解代码”会把scaled dot-product attention作为独立函数封装，并总是显式除以 $\sqrt{d_k}$ 。 31

```
flowchart TB
    X[输入X] --> Q["Q = X W^K"]
    X --> K["K = X W^K"]
    X --> V["V = X W^V"]
    Q --> S["S = Q K^T / sqrt(d_k)"]
    K --> S
    S --> M["+ Mask (optional)"]
    M --> P["P = softmax(S)"]
    P --> O["O = P V"]
    V --> O
```

图2：缩放点积注意力（Scaled Dot-Product Attention）的计算流程：线性投影→打分→可选掩码→softmax→加权求和。 29

掩码机制：同时解决“padding无效token”和“自回归不看未来”

论文在解码器自注意力中引入mask，防止位置 i 关注未来位置 $> i$ ，并配合“输出embedding右移一位”保证自回归生成正确性。 24

工程上通常还有另一类mask：**padding mask**（屏蔽补齐token），以避免不同长度样本batch化时padding参与注意力。Hugging Face文档把attention mask作为通用概念：它指示哪些token应该被attend、哪些不该。 32

把mask并入attention最常见的方式，是在softmax前把非法位置的score置为 $-\infty$ （或一个足够小的负数，如 $-1e9$ ），使softmax后对应权重为0。Harvard NLP的Annotated Transformer给出了典型实现：
`scores.masked_fill(mask == 0, -1e9)`。 31

形式化写法：

- 令原始打分矩阵 $S = \frac{QK^\top}{\sqrt{d_k}}$ 。 33
- 给定mask矩阵 $M \in \{0, 1\}^{n \times n}$ ，定义

$$S'_{ij} = \begin{cases} S_{ij}, & M_{ij} = 1 \\ -\infty, & M_{ij} = 0 \end{cases}$$

• 则

$$A = \text{softmax}(S'), \quad \text{Attention}(Q, K, V) = AV.$$

这与主流实现逻辑一致。 34

Multi-Head Attention：为什么“分头”能提升表达力

论文指出：与其在全维度上做一次注意力，不如把 Q, K, V 做 h 组不同的线性投影，在较低维子空间并行做注意力，再concat回去，最后经 W^O 映射回 d_{model} 。相应公式是：

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \end{aligned}$$

论文并明确给出各投影矩阵的形状与base配置（ $h = 8$ ，每头64维）。 30

论文给出的直觉是：多头允许模型在**不同表示子空间、不同位置关系**上同时“关注”，而单头注意力在某些情况下会被平均效应抑制表达能力。 30

下面给出“可直接照着写”的伪代码（框架无关，强调张量形状与顺序）：

```
# Scaled Dot-Product Attention
# 输入：
#   Q: [B, H, Tq, d_k]
#   K: [B, H, Tk, d_k]
#   V: [B, H, Tk, d_v]
#   mask: 可广播到 [B, 1 or H, Tq, Tk], 1=可见, 0=屏蔽
# 输出：
#   O: [B, H, Tq, d_v]

function SDPA(Q, K, V, mask=None):
    scores = matmul(Q, transpose(K)) / sqrt(d_k) # [B,H,Tq,Tk]
    if mask != None:
        scores = where(mask==0, -INF, scores)
    A = softmax(scores, dim=last) # [B,H,Tq,Tk]
    O = matmul(A, V) # [B,H,Tq,d_v]
    return O, A
```

```
# Multi-Head Attention
# 输入 Xq, Xk, Xv: [B, T, d_model]
function MultiHeadAttention(Xq, Xk, Xv, mask=None):
    Q = linear(Xq, WQ) # [B, Tq, H*d_k]
    K = linear(Xk, WK) # [B, Tk, H*d_k]
    V = linear(Xv, WV) # [B, Tk, H*d_v]

    Q = reshape(Q, [B, H, Tq, d_k])
    K = reshape(K, [B, H, Tk, d_k])
    V = reshape(V, [B, H, Tk, d_v])
```

```

0_heads, A = SDPA(Q, K, V, mask)           # [B,H,Tq,d_v]
O = concat(0_heads over H)                 # [B,Tq,H*d_v]
O = Linear(O, W0)                           # [B,Tq,d_model]
return O, A

```

注意：上面的“reshape/transpose顺序”是多头实现最常出错的地方之一。Annotated Transformer与其issue讨论中，对“为什么要transpose再transpose回来（为了把head维放到batch后，便于批量矩阵乘）”有很多实践细节。²⁵

位置编码：公式、直觉与“相对位移可线性表示”推导

论文的固定位置编码定义为：

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

其中 pos 是位置索引， i 是维度索引；波长从 2π 到 $10000 \cdot 2\pi$ 按几何级数变化。²⁵

论文选择正弦/余弦而非可学习位置embedding的一个核心理由是：它可能让模型更容易学习“按相对位置关注”，因为对任意固定偏移 k ， PE_{pos+k} 可以表示为 PE_{pos} 的线性函数（利用三角恒等式）。²⁵

把这个“线性可表示”写成更具体的推导：

令 $\omega_i = 1/10000^{2i/d_{\text{model}}}$ 。则第 $2i$ 与 $2i+1$ 维分别是 $\sin(pos \cdot \omega_i)$ 与 $\cos(pos \cdot \omega_i)$ 。对偏移 k 有：

$$\begin{aligned} \sin((pos+k)\omega) &= \sin(pos\omega) \cos(k\omega) + \cos(pos\omega) \sin(k\omega), \\ \cos((pos+k)\omega) &= \cos(pos\omega) \cos(k\omega) - \sin(pos\omega) \sin(k\omega). \end{aligned}$$

因此在每对 $(\sin, \cos)$ 维度上， $[\sin((pos+k)\omega), \cos((pos+k)\omega)]^\top$ 可以由 $[\sin(pos\omega), \cos(pos\omega)]^\top$ 通过一个只依赖 k 的二维线性变换得到；这就是论文“相对位移可线性表示”的数学依据。²⁵

复杂度与并行化优势

论文用一张表（Table 1）从三方面对比了不同层类型：**每层复杂度**、**最少串行操作数（并行性）**、**最大路径长度（长程依赖传播距离）**。其核心结论是：**全局自注意力的串行深度为 $O(1)$** ，**最大路径长度 $O(1)$** ，而RNN的两者都是 $O(n)$ 。²⁵

下面给出“Transformer vs RNN vs Conv”的对照表（在论文符号下： n =序列长， d =表示维度， k =卷积核宽度， r =restricted attention邻域大小）。数据取自原论文Table 1。²⁵

维度	Transformer (Self-Attention)	RNN (Recurrent)	CNN (Convolutional)
并行性（样本内）	高：最少串行操作 $O(1)$	低：最少串行操作 $O(n)$	高：最少串行操作 $O(1)$
每层时间复杂度	$O(n^2 \cdot d)$	$O(n \cdot d^2)$	$O(k \cdot n \cdot d^2)$

维度	Transformer (Self-Attention)	RNN (Recurrent)	CNN (Convolutional)
长程依赖路径长度	$O(1)$	$O(n)$	$O(\log_k n)$ (扩张卷积) / $O(n/k)$ (普通堆叠)
长序列代价	二次增长 (瓶颈)	线性增长但难并行	线性增长 (与k相关) 但需堆叠扩感受野

表：Transformer、RNN、CNN在并行性、复杂度与长程依赖路径上的典型差异（来自原论文的理论比较）。

25

论文还给出一个实践性观察：当序列长度 n 小于表示维度 d 时，自注意力层在计算复杂度上可能比循环层更快；并指出在当时机器翻译常用的word-piece/BPE表示中，这个条件往往成立。 36

需要同时看到取舍：全局自注意力的核心瓶颈是 n^2 注意力矩阵带来的时间/显存二次增长，因此论文也提出了“restricted self-attention”（只看邻域 r ）作为处理超长序列的方向，复杂度可降为 $O(r \cdot n \cdot d)$ ，但最大路径长度会变为 $O(n/r)$ 。 37

训练细节与实验复现要点

数据集、分词与batching

论文训练设置（机器翻译部分）非常具体：

- WMT14英德：约 **450万**句对；使用**BPE**，源/目标共享词表约 **37000** token；
- WMT14英法：约 **3600万**句；使用 **32000** word-piece词表；
- batch按句长近似分桶，每个batch约包含 **25000** 源token与 **25000** 目标token。 38

这些细节对复现极其关键：分词方案与词表大小不仅影响BLEU，也影响困惑度（论文在Table 3中强调其per-wordpiece perplexity不可与per-word perplexity直接比）。 39

模型超参（base / big）与消融线索

论文展示了base模型的关键结构超参： $N = 6$ 、 $d_{\text{model}} = 512$ 、 $d_{\text{ff}} = 2048$ 、 $h = 8$ 、 $d_k = d_v = 64$ 、dropout=0.1、label smoothing=0.1，并给出参数量约6500万。 40

big模型对应增大： $d_{\text{model}} = 1024$ 、 $d_{\text{ff}} = 4096$ 、 $h = 16$ ，dropout在Table 3底行标为0.3（但对英法big训练，论文又说明使用0.1而非0.3）。 40

论文的消融（Table 3）对复现调参有两条“可操作经验”：

- key/query维度 d_k 过小会显著损伤效果；
- dropout对防止过拟合很重要；
- learned positional embedding与sinusoidal在该设置下效果接近。 41

优化器、学习率调度与正则化

优化器：Adam， $\beta_1 = 0.9$ ， $\beta_2 = 0.98$ ， $\epsilon = 10^{-9}$ 。 42

学习率调度（公式(3)）：

$$\text{lrate} = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5}), \quad \text{warmup} = 4000.$$

其行为是：前4000步线性升温，之后按 $\text{step}^{-0.5}$ 衰减。 43

正则化：论文在“Regularization”中列出三类（其中最常用的两项是dropout与label smoothing）：

- residual dropout：对每个子层输出加到残差前做dropout，同时对embedding与positional encoding相加后的和也做dropout；base中 $P_{drop} = 0.1$ 。 44

- label smoothing： $\epsilon_{ls} = 0.1$ ，论文指出会“hurt perplexity but improve accuracy and BLEU”。 45

label smoothing的经典更一般定义可追溯到Szegedy等关于Inception的工作（把one-hot标签分布“抹平”到 $(1 - \epsilon)$ 与 ϵ/K 的混合）。 46

训练硬件与推理设置（beam search等）

论文报告训练硬件为单机 **8块NVIDIA 47 P100**：base每步约0.4秒，共训练100k步约12小时；big每步约1秒，训练300k步约3.5天。 48

推理时使用beam search：beam size=4、length penalty $\alpha = 0.6$ ，最大输出长度设置为“输入长度+50”，并使用checkpoint averaging（base平均最后5个checkpoint，big平均最后20个checkpoint）。 49

结果指标与“版本差异”提示

在WMT14英德newstest2014上，论文一致报告big模型 **28.4 BLEU**。 50

英法newstest2014的big模型BLEU在不同发布版本中存在差异：arXiv HTML版本展示为 **41.8**，而NeurIPS PDF版本Table 2为 **41.0**（这会直接影响“你复现到多少算对”）。复现时建议明确对齐“使用的论文版本、数据预处理与评测脚本”。 51

从官方实现角度，Tensor2Tensor仓库给出了可运行的训练/解码流程，并提示在WMT En-De上训练到约28 BLEU的经验值（与论文同量级），这经常被用作工程复现实验的基准参照。 52

常见实现细节与工程注意事项

本节聚焦“自己从零实现时最容易踩坑的点”，并尽量指向权威实现/文档。

掩码的两类语义与形状广播

实践中至少要同时支持：

- **padding mask**：屏蔽padding token，避免其参与注意力（常见输入形状为 $[B, T]$ 的0/1向量，再扩展/广播到attention score形状）； 32

- **causal mask (look-ahead mask)**：解码器自回归屏蔽未来token（典型为下三角矩阵）；在有KV cache时，mask还要考虑“历史key/value长度”，否则会出现越界或可见性错误。 53

有些框架会把2D mask转换成4D（例如 $[B, 1, T_q, T_k]$ 或 $[B, H, T_q, T_k]$ ），以统一处理多头与不同attention后端。Hugging Face在attention interface与相关工具代码中对mask格式转换给出了明确接口与异常提示。 54

```
graph LR
    t1((1)) --> t1
    t2((2)) --> t1
    t2 --> t2
    t3((3)) --> t1
    t3 --> t2
    t3 --> t3
```



```

t4((4)) --> t1
t4 --> t2
t4 --> t3
t4 --> t4

```

图3：因果掩码（causal mask）的可见性示意：位置 i 只能关注 $\leq i$ 的token（自回归生成）。⁵⁵

数值稳定性：softmax前“减最大值”、mask用 $-1e9$ 而非NaN

Scaled dot-product attention的scores可能较大；工程实现通常会在softmax内部做数值稳定处理（例如减去最后一维的最大值），mask位置用非常小的负数近似 $-\infty$ ，避免`exp(-inf)`导致NaN传播。Annotated Transformer展示了经典写法`masked_fill(mask == 0, -1e9)`。³¹

多头张量排布：transpose/contiguous与性能

多头注意力常把张量reshape为 $[B, H, T, d_k]$ 以便做批量矩阵乘；如果实现语言/框架对内存连续性敏感，transpose后需要`contiguous`或等价操作，否则可能出现错误或显著降速。Annotated Transformer的实现与讨论issue中就集中在这些“看似细节但影响巨大”的张量操作上。³⁵

解码加速：KV cache与beam search的“形状地狱”

论文在推理中使用beam search与长度惩罚，并给出beam size与 α 等超参。⁵⁶

当你实现beam search时，几乎必然要引入KV cache：把各层历史的 K, V 缓存起来，避免每生成一个token都对整段前缀重复计算注意力。这会把注意力从“ $[T, T]$ 全矩阵”变成“ $[1, T]$ 对历史的查询”，但同时引入“历史长度=已生成长度 $\times$ beam”的复杂索引与mask生成问题。Hugging Face在工具代码中显式要求传入`key_value_length`来正确创建因果mask，正是为了处理cache情形。⁵⁷

Post-LN vs Pre-LN：训练稳定性与warmup的关系

原论文是Post-LN：LayerNorm($x + \text{Sublayer}(x)$)。²⁴

后续研究指出：Post-LN在初始化时某些层梯度可能偏大，因此需要学习率warmup来稳定训练；而把LayerNorm移到残差分支内部（Pre-LN）可改善梯度行为，并可能降低对warmup的依赖。⁵⁸

从“复现原论文”角度：建议先严格按Post-LN+warmup实现；从“工程训练更稳”角度：可以在确认baseline正确后再尝试Pre-LN（但要清楚：这已不再是原论文的精确复现配置）。⁵⁹

后续改进、应用与局限、学习资源与实践建议

后续改进与衍生工作综述

Transformer之后，研究与工业界基本沿两条主线扩展：

主线A：预训练范式与架构裁剪（Encoder-only / Decoder-only / Encoder-Decoder）

- BERT：以Transformer编码器为骨干，通过“masked language modeling”等目标学习双向表征，并在多项NLP任务上取得强结果。其代表论文作者包含Jacob Devlin⁶⁰、Ming-Wei Chang⁶¹、Kenton Lee⁶²、Kristina Toutanova⁶³。⁶⁴

- GPT（以GPT-1/GPT-2为代表）：以Transformer解码器堆叠为骨干，使用自回归语言建模进行生成式预训练。GPT-1论文作者包含Alec Radford⁶⁵、Karthik Narasimhan⁶⁶、Tim Salimans⁶⁷、Ilya Sutskever⁶⁸，机构为OpenAI⁶⁹。⁷⁰

(如果你面向工程微调任务, Hugging Face对“causal vs masked language modeling”的任务定义与训练方式也提供了清晰说明, 可帮助你把架构与训练目标对应起来。) ⁷¹

主线B: 长序列与高效注意力(缓解 n^2 瓶颈)

- Transformer-XL: 针对语言模型“固定上下文窗口”的限制, 引入segment-level recurrence与新的位置编码方案, 用于更长依赖建模; 其论文作者包含Zihang Dai ⁷²、Zhilin Yang ⁷³、Yiming Yang ⁷⁴、Jaime Carbonell ⁷⁵、Quoc V. Le ⁷⁶、Ruslan Salakhutdinov ⁷⁷。 ⁷⁸
- Longformer: 指出标准self-attention二次复杂度导致长文档不可用, 提出“局部滑窗+全局token”的稀疏注意力模式, 把复杂度降到线性量级, 并给出Longformer-Encoder-Decoder (LED) 支持长文档生成; 论文作者包含Iz Beltagy ⁷⁹、Matthew E. Peters ⁸⁰、Arman Cohan ⁸¹。 ⁸²
- Sparse Transformers (Generating Long Sequences with Sparse Transformers): 通过稀疏分解attention矩阵, 把注意力从 $O(n^2)$ 降到 $O(n\sqrt{n})$ 并探索超长序列建模。 ⁸³
- Reformer: 用LSH近似注意力把复杂度降到 $O(L \log L)$, 并用可逆残差减少激活存储。 ⁸⁴
- Linformer: 假设注意力矩阵可低秩近似, 把时间与空间复杂度降为线性。 ⁸⁵
- Performer: 用随机特征近似softmax attention核, 给出线性复杂度的近似注意力并带有理论保证。 ⁸⁶

这些工作与原论文的“restricted self-attention”设想在问题意识上是一脉相承的: 当序列足够长, **全连接注意力矩阵**成为瓶颈, 需要在稀疏性、低秩性或核近似上做结构化约束。 ⁸⁷

应用场景与局限性

典型应用: 机器翻译、文本生成、摘要、抽取式/生成式问答、序列标注、代码建模等。Transformer最初在机器翻译与句法分析上验证了通用性。 ⁸⁸

在工程上, encoder-decoder形态广泛用于翻译/摘要, decoder-only形态用于生成式语言模型, encoder-only形态用于检索、分类与表征学习。 ⁸⁹

局限性(对原版Transformer尤其重要):

- 长序列成本高: 标准self-attention随序列长度二次扩展, 使长文档/长时间序列训练与推理困难, 这是Longformer等工作的直接动机。 ⁸²
- 自回归解码串行: 虽然训练端可并行 (teacher forcing下同时预测多位置), 但推理端逐token生成不可避免地串行, 论文末尾也点到“Making generation less sequential”是未来方向。 ⁹⁰
- 训练稳定性依赖技巧: 原论文的Post-LN结构与学习率warmup强绑定, 后续关于LayerNorm位置的研究显示其对梯度与稳定性很关键。 ⁹¹

优先参考来源与推荐代码仓库

按你提出的“优先级”给出推荐(尽量选择原始论文/官方实现/公认高质量讲解):

- 原论文 (arXiv HTML/NeurIPS PDF两个版本都建议保存, 以便核对数值差异)。 ¹
- 官方实现 (Tensor2Tensor, 包含训练/解码命令与可复现实验建议)。 ⁹²
- 高质量讲解与对照实现:
- The Annotated Transformer (强烈推荐: 逐段解释+参考实现, 尤其适合核对mask/张量形状/attention细节)。 ⁹³
- 《动手学深度学习》中文章 (适合系统梳理概念、公式与实现思路)。 ²⁷
- Hugging Face ⁹⁴ Transformers (训练任务定义、mask语义、工程化生态完善, 适合做可运行实验与模型对照)。 ⁹⁵
- Meta ⁹⁶ 系的fairseq Transformer实现 (NMT与research-friendly配置较多, 利于对照不同实现细节)。 ⁹⁷
- OpenNMT (提供Transformer模型与mask实现源码, 适合做端到端翻译系统复现/部署对照)。 ⁹⁸

实践建议：如何从零实现一个小型Transformer

下面给出“从零可跑通、可调试、可逐步扩大”的工程路线图（不绑定框架）。

建议的最小可用模型（toy → small）

- 层数：encoder/decoder各2-4层（先从2层开始，确认训练收敛与mask正确，再加深）。思路来自原论文N=6的可扩展堆叠方式。²¹
- 维度： $d_{\text{model}} = 128$ 或256； $h = 4$ 或8； $d_{\text{ff}} \approx 4d_{\text{model}}$ （原论文base是512→2048）。⁴¹
- dropout：0.1起步（与原论文base一致）。⁴³
- label smoothing：0.1（与原论文一致；如果任务很小可先关掉以便更快过拟合验证）。⁹⁹
- 优化器与LR：先严格照搬Adam参数与warmup调度，能显著减少“训练不动”的概率。⁴²

训练数据量与任务选择

- 若目的是验证实现正确：选择“可快速收敛、可过拟合”的小数据（例如几千到几万句对的toy翻译/复制任务），先确保loss能降到很低，且解码输出与期望一致。该思路与多数参考实现“先小规模跑通再扩展”一致。¹⁰⁰
- 若目的是接近论文设置：对照WMT14 En-De的BPE与batch token数等关键设置；Tensor2Tensor仓库的problem与hparams名能减少数据预处理偏差。¹⁰¹

调试技巧（强烈建议做成单元测试）

- 1) **形状断言**：每一步都打印/断言 $[B, H, T, d_k]$ 、 $[B, H, T, T]$ 等关键张量；多头reshape错一次，模型可能“还能跑但学不到”。³⁵
- 2) **mask可视化**：对一个短序列（T=4或8）输出attention权重矩阵，检查：
 - causal mask是否严格下三角；
 - padding位置是否权重 ≈ 0 。¹⁰²
- 3) **过拟合一条样本**：固定1个batch反复训练，确认能趋近0损失；如果不行，优先查mask、位置编码加法、LayerNorm位置、学习率调度。¹⁰³

常见陷阱与解决方法

- 忘记除以 $\sqrt{d_k}$ ：表现为训练早期不稳定或收敛很慢；检查attention实现。²⁹
- mask写反（0/1语义颠倒）或广播维度错误：表现为模型在解码时“偷看未来”或padding干扰；用短序列可视化最有效。¹⁰⁴
- Post-LN但学习率过大/无warmup：表现为loss爆炸或NaN；按原论文加warmup，或改为Pre-LN（但这会偏离原论文复现）。¹⁰⁵
- 评测不可比：BLEU对tokenization与脚本敏感；复现时要记录“使用的论文版本（41.8 vs 41.0）+数据预处理+评测配置”。¹⁰⁶

附录：初学者背景概念速查（简版）

（若你已熟悉神经网络训练与RNN/CNN，可跳过。）

- **Softmax**：把实数向量 $\{s_j\}$ 映射为概率分布 $\alpha_j = \exp(s_j) / \sum_k \exp(s_k)$ 。在注意力中用来把“相似度分数”转为归一化权重。²⁹
- **残差连接（Residual Connection）**：输出写成 $x + \text{Block}(x)$ ，帮助深层网络训练（最早在ResNet中系统化）。¹⁰⁷
- **层归一化（LayerNorm）**：对每个样本、每个位置的特征维做归一化，提升训练稳定性；Transformer原版使用LayerNorm配合残差。¹⁰⁸
- **BPE/WordPiece**：子词分词方法，把稀有词拆成高频子词，缓解OOV并控制词表大小；原论文英德用BPE约37k token，英法用32k word-piece。³⁸

- **Label Smoothing**: 把one-hot标签分布稍微“抹平”，降低过度自信，常提升泛化与BLEU；论文使用 $\epsilon_{ls} = 0.1$ 。 ⁹⁹
- **BLEU**: 机器翻译常用自动评测指标，基于n-gram精确率与长度惩罚；对tokenization非常敏感，因此复现需对齐评测流程。 ¹⁰⁹

1 17 20 51 68 90 106 Attention Is All You Need

<https://arxiv.org/html/1706.03762v7>

2 8 9 31 34 35 69 72 The Annotated Transformer

<https://nlp.seas.harvard.edu/2018/04/03/attention.html>

3 4 5 6 10 11 14 15 18 19 21 22 23 24 25 26 28 29 30 33 36 37 38 39 40 41 42 43 44
45 48 49 50 55 56 60 61 62 63 65 66 73 74 76 79 87 94 96 99 101 102 103 104 105 107 109

Attention is All you Need

<https://papers.neurips.cc/paper/7181-attention-is-all-you-need.pdf>

7 52 GitHub - tensorflow/tensor2tensor: Library of deep learning models and datasets designed to make deep learning more accessible and accelerate ML research. • GitHub

<https://github.com/tensorflow/tensor2tensor>

12 92 tensorflow/tensor2tensor: Library of deep learning models ...

https://github.com/tensorflow/tensor2tensor?utm_source=chatgpt.com

13 58 59 91 On Layer Normalization in the Transformer Architecture

https://arxiv.org/abs/2002.04745?utm_source=chatgpt.com

16 64 89 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

https://arxiv.org/abs/1810.04805?utm_source=chatgpt.com

27 10.7. Transformer — 动手学深度学习2.0.0 documentation

https://zh.d2l.ai/chapter_attention-mechanisms/transformer.html?utm_source=chatgpt.com

32 75 Glossary

https://huggingface.co/docs/transformers/glossary?utm_source=chatgpt.com

46 Rethinking the Inception Architecture for Computer Vision

https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Szegedy_Rethinking_the_Inception_CVPR_2016_paper.pdf?utm_source=chatgpt.com

47 81 108 Layer Normalization

https://arxiv.org/abs/1607.06450?utm_source=chatgpt.com

53 57 modeling_attn_mask_utils.py - huggingface/transformers

https://github.com/huggingface/transformers/blob/main/src/transformers/modeling_attn_mask_utils.py?utm_source=chatgpt.com

54 Attention backends

https://huggingface.co/docs/transformers/en/attention_interface?utm_source=chatgpt.com

67 95 Transformers: the model-definition framework for state-of- ...

https://github.com/huggingface/transformers?utm_source=chatgpt.com

70 Improving Language Understanding by Generative Pre- ...

https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf?utm_source=chatgpt.com

71 Causal language modeling

https://huggingface.co/docs/transformers/en/tasks/language_modeling?utm_source=chatgpt.com

77 84 Reformer: The Efficient Transformer

https://arxiv.org/abs/2001.04451?utm_source=chatgpt.com

78 80 Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context

https://arxiv.org/abs/1901.02860?utm_source=chatgpt.com

82 Longformer: The Long-Document Transformer

https://arxiv.org/abs/2004.05150?utm_source=chatgpt.com

83 Generating Long Sequences with Sparse Transformers

https://arxiv.org/abs/1904.10509?utm_source=chatgpt.com

85 Linformer: Self-Attention with Linear Complexity

https://arxiv.org/abs/2006.04768?utm_source=chatgpt.com

86 [2009.14794] Rethinking Attention with Performers

https://arxiv.org/abs/2009.14794?utm_source=chatgpt.com

88 Attention Is All You Need

https://arxiv.org/abs/1706.03762?utm_source=chatgpt.com

93 The Annotated Transformer

https://nlp.seas.harvard.edu/2018/04/03/attention.html?utm_source=chatgpt.com

97 fairseq/fairseq/models/transformer/transformer_base.py at ...

https://github.com/pytorch/fairseq/blob/main/fairseq/models/transformer/transformer_base.py?utm_source=chatgpt.com

98 Transformer — OpenNMT-tf 2.32.0 documentation

https://opennmt.net/OpenNMT-tf/package/opennmt.models.Transformer.html?utm_source=chatgpt.com

100 The Annotated Transformer

https://aclanthology.org/W18-2509.pdf?utm_source=chatgpt.com